Mock 3 — LLM Inference Engine

ICF house style, 4 levels, 200-600

LLM Inference Engine — Coding Assessment

Your task is to implement a simplified LLM inference serving engine. All operations that should be supported are listed below. Partial credit will be granted for each test passed, so run `python tests.py` often to receive partial credit for passed tests. Please check tests for requirements and argument types.

Implementation Tips Read the question all the way through before you start coding, but implement the operations and complete the levels one by one, not all together, keeping in mind that you will need to refactor to support additional functionality. Please, do not change the existing method signatures.

Time limit: **90 minutes**. Score: 200–600.

Background

A serving engine processes inference requests in two phases. **Prefill** ingests the prompt, processing `PREFILL_RATE = 4` tokens per tick. **Decode** then generates output tokens, one per tick.

The **cost** of a request is the total ticks it needs:

```
cost = ceil(prompt_tokens / 4) + max_tokens
```

Level 1 — Initial Design & Basic Functions

- `submit(request_id, prompt_tokens, max_tokens)`
  - Register a request. Returns `True` on success.
  - If a request with that id already exists, returns `False` and changes nothing.
- `get_cost(request_id)`
  - Return the request's cost, or `None` if no such request exists.
- `cancel(request_id)`
  - Remove the request. Returns `True` if it existed, `False` otherwise.

Level 2 — Data Structures & Data Processing

- `total_cost()`
  - Sum of the costs of all registered requests.
- `top_n_costly(n)`
  - Return a list of strings `"<request_id>(<cost>)"` for the `n` most expensive requests, ordered by cost descending, ties broken by `request_id` ascending.
  - If fewer than `n` requests exist, return all of them.

Level 3 — Refactoring & Encapsulation

Requests may now have a limited lifespan — a client that has given up should stop consuming engine state. Implement timestamped variants of the existing operations. These **inherit all functionality** of the originals and additionally take a `timestamp`; `submit_at` may specify a `ttl`. No ttl means the request lives forever.

A request submitted at time `t` with ttl `x` is visible to queries with timestamp in `[t, t + x)`. At `t + x` it has expired and must be invisible to every operation.

- `submit_at(timestamp, request_id, prompt_tokens, max_tokens)`
- `submit_at(timestamp, request_id, prompt_tokens, max_tokens, ttl)`
  - An id whose previous request has expired may be reused.
- `get_cost_at(timestamp, request_id)`
- `cancel_at(timestamp, request_id)`
- `total_cost_at(timestamp)`
- `top_n_costly_at(timestamp, n)`

The Level 1 and Level 2 methods must keep working exactly as before.

Level 4 — Extending Design & Functionality

The engine now serves requests across multiple workers.

- `add_worker(worker_id)`
  - Register a worker. Returns `True`, or `False` if that worker already exists.
- `assign_at(timestamp)`
  - Assign every request that is alive at `timestamp` to a worker, using **least-loaded** balancing: a worker's load is the total cost of the requests already assigned to it during this call, starting from zero.
  - Process requests in order of `(submission time, request_id)`. For each, choose the worker with the smallest load; break ties by `worker_id` ascending (lexicographic).
  - Return a dict mapping `request_id -> worker_id`. If no workers are registered, return an empty dict.
- `worker_load_at(timestamp, worker_id)`
  - Total cost of the alive requests currently assigned to that worker, or `None` if the worker isn't registered.

Notes

- Standard library only.
- Every ordering rule above is exact — determinism is graded.
- The grader is `tests.py`; implement in `engine.py`.